
NOTE DE CADRAGE PROJET

Refonte de la Plateforme de Gestion des Goodies

Fromagerie DigiCheese

Référence	NC-DigiCheese-v1.0
Date de création	Avril 2026
Version	V1.0
Statut	En cours - à faire évoluer
Équipe projet	Julien Milhau Villar Thibault Odor Victor Odin
Commanditaire	Fromagerie DigiCheese
Référent DSI	Christophe Germain - cgermain@diginamic.fr

1. Contexte et origine du projet	3
1.1 Historique et situation actuelle	3
1.2 Problématique identifiée	3
2. Enjeux et objectifs du projet	3
2.1 Objectif principal	3
2.2 Objectifs spécifiques	3
2.3 Résultats attendus	4
2.4 Présentation de la cible	4
3. Périmètre du projet	5
3.1 Ce qui est dans le périmètre	5
3.2 Ce qui est hors périmètre (V1)	5
3.3 Livrables identifiés	6
4. Parties prenantes	7
4.1 Équipe projet	7
4.2 Commanditaire et utilisateurs finaux	7
5. Planning et jalons	8
5.1 Jalons principaux	8
5.2 Planning prévisionnel	9
5.2 Méthode de travail	9
6. Architecture technique retenue	10
6.1 Vue d'ensemble	10
6.2 Stack technique	10
7. Sécurité et contraintes	11
7.1 Authentification et autorisations	11
7.2 Mesures de sécurité applicative	11
7.3 Conformité RGPD	11
8. Risques et contraintes identifiés	12
9. Moyens mis à disposition	13
9.1 Moyens humains	13
9.2 Moyens financiers	13
9.3 Moyens techniques	13
10. Budget prévisionnel	14
10.1 Coûts de développement	14
10.2 Coûts d'infrastructure mensuelle	14
10.3 Récapitulatif global	15
11. Gestion documentaire et versioning	16
11.1 Historique des versions de cette note	16
11.2 Documents de référence	16
11.3 Évolutions prévues de cette note	16
12. Circuit de validation	17

1. Contexte et origine du projet

1.1 Historique et situation actuelle

La Fromagerie DigiCheese est une TPE spécialisée dans la vente de produits fromagers et la fidélisation de sa clientèle via un programme de cadeaux (goodies). Depuis plus de vingt ans, la gestion de ce programme repose sur une application développée sous Microsoft Access 2000 en VBA.

Cette application présente aujourd'hui de nombreuses limitations bloquantes :

- Forte instabilité (bugs réguliers, crashes fréquents)
- Non supportée officiellement depuis Windows Vista
- Support éditeur terminé (standard en 2004, étendu en 2009)
- Interface rigide, inesthétique et peu ergonomique
- Faibles possibilités d'évolution ou de maintenance
- Client lourd peu performant au regard des technologies actuelles

1.2 Problématique identifiée

Le système actuel ne permet plus d'assurer une gestion efficace et fiable des commandes de cadeaux fidélité. L'outil est devenu un frein opérationnel pour les équipes par sa rigidité et l'impossibilité de la faire évoluer. Une refonte complète du système d'information s'impose pour moderniser, sécuriser et pérenniser cette activité.

2. Enjeux et objectifs du projet

2.1 Objectif principal

Remplacer l'application Access 2000 par une API backend moderne, accessible via une interface web interne, capable de gérer l'ensemble du cycle de vie des commandes de cadeaux fidélité : saisie des clients, gestion des commandes, expédition des colis, gestion des stocks, et administration des paramètres métier.

2.2 Objectifs spécifiques

- Moderniser l'architecture technique avec une stack Python / Flask / PostgreSQL
- Exposer une API REST complète et documentée (Swagger/OpenAPI)
- Sécuriser les accès par un système d'authentification et de rôles (RBAC)

- Améliorer l'ergonomie et l'accessibilité pour les opérateurs non-techniciens
- Garantir la conformité RGPD dans le traitement des données clients
- Mettre en place un pipeline CI/CD pour pérenniser la maintenabilité

2.3 Résultats attendus

- Sprint 1 (livré) : Backend API REST opérationnel avec CRUD complet sur tous les modules métier, authentification, gestion des rôles et tests pytest passants
- Sprint 2 (à venir) : Interface front-end Vue.js connectée à l'API et CRUD complet pour les opérateurs stock
- Production : Déploiement sur infrastructure sécurisée avec supervision Grafana / Prometheus

2.4 Présentation de la cible

L'environnement souhaité après réalisation du projet se décompose de la façon suivante:

- Un environnement de production: intranet accessible via un frontal web permettant au client de gérer, en fonction du rôle de chacun, gérer l'envoi de goodies notamment à l'aide d'un système de colisage automatique, gérer les utilisateurs de la plateforme, et gérer les stocks.
- Un environnement de préproduction: un réplica aussi proche que possible techniquement de l'environnement de production. Il servira au client de site de recettage afin de s'assurer que chaque nouveau développement est conforme aux attentes.
- Un environnement de développement: il permettra aux développeurs un accès facilité à la base de code et de travailler dans un environnement sain.
- Une CI/CD: celle-ci aura vocation à s'assurer que les tests sont passant ainsi qu'à formater et linter le code de façon à assurer une pérennité et une qualité du code dans le temps.

3. Périmètre du projet

3.1 Ce qui est dans le périmètre

L'application est structurée autour de trois modules fonctionnels correspondant aux rôles utilisateurs :

Administration (rôle Admin)

- Gestion des comptes utilisateurs (CRUD)
- Gestion des référentiels métier : communes, objets/goodies, conditionnements
- Accès à la documentation API (Swagger UI)

Gestion des colis (rôle OP-colis)

- Gestion des clients et adresses (CRUD)
- Gestion des commandes et détails de commande (CRUD)
- Génération de template de mailing client
- Consultation des statistiques d'activité

3.2 Ce qui est hors périmètre (V1)

- Gestion des points de fidélité clients (non implémentée dans le backend actuel)
- Interface front-end Vue.js (prévue Sprint 2)
- Envoi d'emails réels (le template de mailing est fourni mais l'envoi SMTP n'est pas intégré)
- Module de reporting avancé ou tableau de bord analytics
- Application mobile
- La gestion des stocks (rôle OP-stocks)

3.3 Livrables identifiés

Livrable	Description	Statut
Backend API REST	API Flask avec CRUD complet sur tous les modules (hors opérateur stock)	Prévu (Sprint 1)
Tests pytest	Tests d'authentification, de rôles et de routes	Prévu (Sprint 1)
Documentation Swagger	Swagger UI auto-générée via Flasgger	Prévu (Sprint 1)
CDCT v3.0	Cahier des Charges Techniques mis à jour	Prévu (Avril 2026)
Backend API REST	API Flask pour opérateur stock avec CRUD complet	Prévu (Sprint 2)
Front-end Vue.js	Interface utilisateur SPA connectée à l'API	Prévu (Sprint 2)
Pipeline CI/CD	GitHub Actions : tests, build, déploiement	Prévu
Infrastructure PROD	Serveurs virtuels, Traefik, Gunicorn, PostgreSQL	Prévu

4. Parties prenantes

4.1 Équipe projet

Nom	Rôle	Contact
Julien Milhau Villar	Développeur - Lead Back-end	
Thibault Odor	Développeur - Architecture & Sécurité	
Victor Odin	Développeur - Base de données & Tests	
Christophe Germain	Chef de Projet / Référent DSI	cgermain@diginamic.fr

4.2 Commanditaire et utilisateurs finaux

Nom	Qualité / Rôle	Contact
Fromagerie DigiCheese	Client principal / commanditaire	contact@digicheese.com
Christophe Germain	Directeur DSI	cgermain@diginamic.fr
Opérateurs colis	Utilisateurs finaux - module colis	Salariés fromagerie
Opérateurs stocks	Utilisateurs finaux - module stock	Salariés fromagerie
Administrateur	Gestion utilisateurs et référentiels	Salarié dédié

5. Planning et jalons

5.1 Jalons principaux

Jalon	Description	Statut / Date
Kick-off projet	Réunion initiale, expression du besoin	Janvier 2026
CDCT V1.0	Première version du cahier des charges	Avril 2026
Sprint 1 - Backend	API REST Flask complète, tests, Swagger	Avril 2026
Note de cadrage V1	Premier cadrage formel du projet	Avril 2026
Sprint 2 - Front-end	Interface Vue.js connectée à l'API	À planifier
Recette / uat	Validation fonctionnelle avec le client	À planifier
Mise en production	Déploiement environnement PROD	À planifier

5.2 Planning prévisionnel

Le diagramme suivant représente la planification des deux sprints sur 11 semaines effectives de développement. Le Sprint 1 couvre la période janvier 2026 à février 2026. Le Sprint 2 se déroule sur 5 semaines consécutives entre avril et mai de la même année.

Planning Global	SPRINT 1 - Back-end API REST Flask						SPRINT 2 - Front -end Vue.js				
Epic / Activité	S1 01-02 Jan	S2 05-09 Jan	S3 12-16 Jan	S4 19-23 Jan	S5 26-30 Jan	S6 02-06 Fev	S7 01-03 Avril	S8 06-10 Avril	S9 13-17 Avril	S10 20-24 Avril	S11 27-01 Avril
S1-1 Infrastructure & BDD											
S1-2 Auth & rôles JWT											
S1-3 Admin CRUD											
S1-4 Colis CRUD											
S1-5 Stock CRUD											
S1-6 Tests, CI/CD, Swagger											
🚩 JALON - API Sprint 1 livrée											
S2-1 Socle Vue.js / Auth front											
S2-2 Interface Admin											
S2-3 Interface OP Colis											
S2-4 Interface OP Stock											
S2-5 Tests Vitest + Cypress											
CI/CD front + déploiement											
🚩 JALON - Livraison PREPROD Sprint 2											

5.2 Méthode de travail

Le projet est conduit en méthode Agile Scrum avec des sprints de deux semaines. Les outils retenus sont JIRA (gestion du backlog et des sprints) et GitHub (versioning, Pull Requests, CI/CD via GitHub Actions).

Cérémonies Scrum : Sprint Planning (2h), Daily Stand-up (15 min), Sprint Review (1h), Sprint Retrospective (45 min), Backlog Refinement (1h).

6. Architecture technique retenue

6.1 Vue d'ensemble

Architecture 3-tiers de type client-serveur :

- Couche présentation : SPA Vue.js 3 avec Vite (Sprint 2)
- Couche métier : API REST Python / Flask 3.1.x (Sprint 1 - livré)
- Couche persistance : Base de données relationnelle PostgreSQL 18

Toutes les communications sont chiffrées en TLS 1.2 minimum. Aucun accès direct à la base de données n'est autorisé depuis le navigateur.

6.2 Stack technique

Composant	Technologie retenue	Justification
Back-end	Python 3.11+ / Flask 3.1.x	Micro-framework léger, adapté à une API REST modulaire
ORM	SQLAlchemy 2.X	Sécurise les requêtes, bonne lisibilité du modèle
Base de données	PostgreSQL 18	Robuste, conforme SQL, adapté aux requêtes complexes
Authentification	Flask-Login (session) =>JWT (Sprint 2)	Session côté serveur pour Sprint 1, migration JWT prévue
Documentation API	Flasgger / Swagger UI	Génération automatique depuis les docstrings Flask
Tests	pytest + pytest-flask	Base Postgres en mémoire, isolation totale de la prod
Front-end (Sprint 2)	Vue.js 3 + Pinia + Axios + Tailwind CSS	SPA moderne, légère, maintenable
Serveur app	Gunicorn 25.x derrière Traefik	Reverse proxy SSL/TLS, Let's Encrypt
CI/CD	GitHub Actions	Déclenchement sur push develop/master
Supervision	Grafana + Prometheus + Sentry	Métriques, logs, remontée erreurs front

7. Sécurité et contraintes

7.1 Authentification et autorisations

- RBAC (Role-Based Access Control) : trois rôles cumulables — Admin, OP-colis, OP-stocks
- Contrôle d'accès côté serveur sur chaque endpoint via décorateur `@role_required`
- Mots de passe hashés avec `bcrypt` (Werkzeug) — migration vers `bcrypt` facteur 12 prévue
- Migration vers JWT (HS256) prévue lors du Sprint 2 avec access token (15-30 min) + refresh token en cookie `HttpOnly/Secure/SameSite`

7.2 Mesures de sécurité applicative

Risque	Mesure de protection
Injection SQL	Utilisation exclusive de SQLAlchemy (requêtes paramétrées)
XSS	Échappement automatique Vue.js + politique CSP via Unicorn
CSRF	JWT dans header Authorization (Sprint 2) + cookie SameSite pour refresh token
Brute force	Limitation des tentatives de connexion (5 max, blocage 15 min)
Données sensibles	Aucune donnée personnelle dans les logs, erreurs génériques côté client
Dépendances vulnérables	Audit pip-audit / npm audit intégré au pipeline CI/CD

7.3 Conformité RGPD

- Minimisation des données collectées
- Droit d'accès, rectification et effacement via l'interface admin
- Données clients conservées 3 ans après le dernier achat, puis anonymisées automatiquement
- Chiffrement TLS en transit, accès base de données restreint au réseau interne
- Journalisation des accès aux données sensibles avec horodatage et identifiant utilisateur

8. Risques et contraintes identifiés

Risque	Probabilité	Impact	Mesure de mitigation
Migration des données depuis Access	Élevée	Élevé	Back-up de l'ancienne BDD avant test d'insertion dans la nouvelle
Disponibilité de l'équipe (formation en parallèle)	Moyenne	Moyen	Sprints courts, revues régulières avec le PO
Évolution du périmètre fonctionnel	Moyenne	Moyen	Backlog priorisé, DoR et DoD définis, validation PO obligatoire
Compatibilité navigateurs (Edge entreprise)	Faible	Faible	Tests manuels sur navigateurs cibles N et N-1
Sécurité - fuite de données	Faible	Élevé	HTTPS obligatoire, RBAC, audit régulier des dépendances
Scalabilité future (croissance données)	Faible	Faible	PostgreSQL + 100 Go stockage secondaire offre large marge

9. Moyens mis à disposition

9.1 Moyens humains

L'équipe projet est composée de trois développeurs aux rôles complémentaires : Julien Milhau Villar, en charge du développement back-end et du leadership technique ; Thibault Odor, responsable de l'architecture et de la sécurité ; et Victor Odin, en charge de la base de données et des tests. L'ensemble de l'équipe intervient dans le cadre d'une formation Lead Dev/DevOps Diginamic 2025-2026. Le projet est piloté par Christophe Germain, référent DSI et chef de projet côté commanditaire. Les utilisateurs finaux — opérateurs colis, opérateurs stocks et administrateurs — seront sollicités lors des phases de recette (UAT) pour valider les développements réalisés. Du côté du commanditaire, la Fromagerie DigiCheese désigne un interlocuteur principal joignable à contact@digicheese.com.

9.2 Moyens financiers

Les choix technologiques retenus (Python, Flask, PostgreSQL, Vue.js, GitHub Actions) sont exclusivement des outils open source, ce qui limite les dépenses liées aux licences logicielles. Les principaux postes de coûts à anticiper concernent l'infrastructure d'hébergement en production (serveurs virtuels, nom de domaine, certificats TLS via Let's Encrypt) ainsi que les éventuels outils de supervision. Ces éléments budgétaires seront formalisés en partie 10 de cette note de cadrage et validés avec le commanditaire avant la mise en production.

9.3 Moyens techniques

Les moyens techniques s'appuient sur une stack entièrement définie et documentée dans le CDCT V3.0. Côté back-end, l'API REST est développée en Python 3.11+ avec Flask 3.1.x, SQLAlchemy 2.x comme ORM et PostgreSQL 18 comme base de données relationnelle. Le front-end (Sprint 2) sera développé en Vue.js 3, avec Pinia, Axios et Tailwind CSS. L'infrastructure cible repose sur Gunicorn comme serveur applicatif, derrière un reverse proxy Traefik gérant le chiffrement TLS. La supervision sera assurée par la combinaison Grafana, Prometheus et Sentry. Le versioning du code est hébergé sur GitHub, avec un pipeline CI/CD via GitHub Actions assurant l'automatisation des tests, du linting et du déploiement. Trois environnements distincts sont prévus : développement, préproduction (recettage) et production. Les outils de gestion de projet retenus sont JIRA pour le suivi du backlog et des sprints, et GitHub pour la gestion des pull requests et la CI/CD.

10. Budget prévisionnel

10.1 Coûts de développement

Le projet se déroule sur deux sprints : le premier de 30 jours ouvrés, le second de 25 jours ouvrés, soit un total de 55 jours ouvrés. Trois développeurs sont mobilisés à temps plein sur l'ensemble de la période. En retenant un Taux Journalier Moyen (TJM) de référence de 450 €/jour pour un développeur junior à intermédiaire en contexte formation/projet (hors charges patronales), le coût total de développement se détaille comme suit :

Élément	Sprint 1	Sprint 2	Total
Durée	30 jours	25 jours	55 jours
Nb développeurs	3	3	3
Jours/homme	90 j/h	75 j/h	165 j/h
TJM unitaire	450 €	450 €	450 €
Coût main d'œuvre	40 500 €	33 750 €	74 250 €

Ces estimations sont basées sur le planning global: 5.2 Planning prévisionnel.

10.2 Coûts d'infrastructure mensuelle

Pour un serveur bare metal de petite taille, hébergé en France, avec les caractéristiques suivantes — 32 Go de RAM, 250 Go de stockage SSD, connectivité 1 Gbps — les principaux acteurs du marché français (OVHcloud, Scaleway, Hetzner FR) proposent des offres dans la fourchette suivante :

Poste	Détail	Coût mensuel estimé
Serveur bare metal	32 Go RAM / 250 Go SSD / datacenter France	80 € – 120 € HT
Certificat TLS	Let's Encrypt (via Traefik)	Gratuit
Nom de domaine	.fr ou .com (amorti sur 12 mois)	1 € – 2 €
Sauvegarde distante	Snapshot ou stockage objet (optionnel)	5 € – 15 €
Total estimé		~86 € – 137 € HT/mois

10.3 Récapitulatif global

Poste	Nature	Montant estimé
Développement (Sprint 1 + 2)	Ponctuel	74 250 € HT
Infrastructure (mensuel, récurrent)	Récurrent	~86 € – 137 € HT/mois
Outils projet (JIRA, GitHub)	Récurrent	Gratuit / faible coût*

11. Gestion documentaire et versioning

11.1 Historique des versions de cette note

Version	Date	Description	Auteur(s)
V1.0	Avril 2026	Première version de la note de cadrage - basée sur le CDCT Sprint 1	Équipe DigiCheese

11.2 Documents de référence

- CDCT — Cahier des Charges Techniques DigiCheese V3.0 (Avril 2026)
- Mini Cahier des Charges Fonctionnels — Refonte SI DigiCheese V2.0 (Octobre 2022)
- Guide Rédiger une note de cadrage — Diginamic / Christophe Germain V1.0 (2022)
- Code source backend — Dépôt GitHub TP7 Diginamic Lead Dev/DevOps 2025-2026

11.3 Évolutions prévues de cette note

Cette note de cadrage est un document vivant. Elle sera mise à jour à chaque jalon significatif du projet, notamment :

- V1.1 - À l'issue du cadrage du Sprint 2 (front-end Vue.js)
- V1.2 - Après validation UAT et avant mise en production
- V2.0 - Lors de l'ajout de tout nouveau module fonctionnel majeur

12. Circuit de validation

Version	Valideur	Qualité	Date	Commentaires
V1.0	Équipe DigiCheese	Développeurs / Auteurs	Avril 2026	Première validation interne
V1.0	Christophe Germain	Référent DSI / Chef de projet	À compléter	Validation commanditaire

Ce document est soumis à la validation du commanditaire avant toute diffusion externe. En cas de désaccord entre les parties sur la finalité, les objectifs ou le périmètre du projet, la présente note de cadrage fait foi.